Candidate Feedback Composer: design (F11)

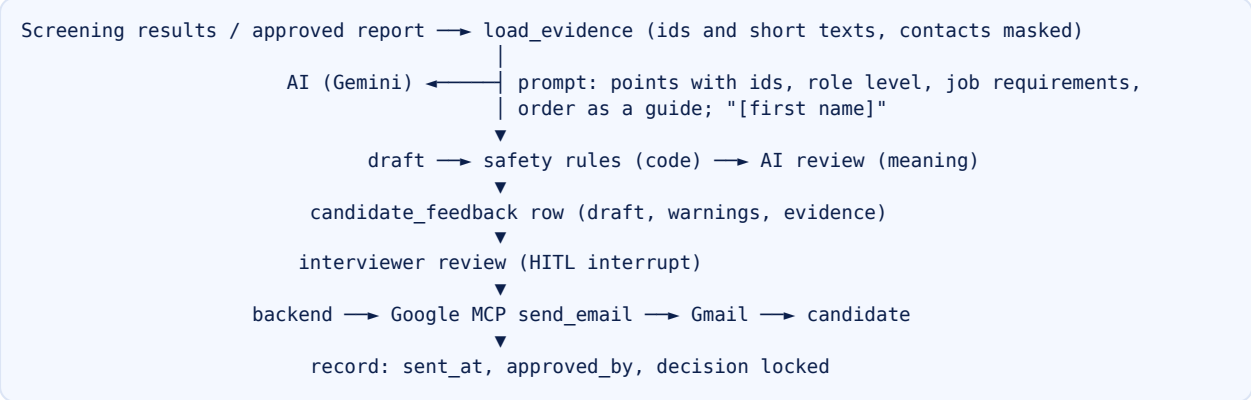
The innovation feature of HireLucent. This document describes the data flow, the workflow, the API and how the feature plugs into the existing modules. The reasons for choosing it are in the innovation rationale (`HireLucent_Innovation_Rationale_30221_Farhan.pdf`); the graph is drawn in `LangGraph_5_Candidate_Feedback_30221_Farhan.pdf` .

1. Who gets which email

Stage	Who	Decision	Email kind	First draft
Screening	Scored but not shortlisted (`screened`)	Rejected automatically	`screening_reject`	AI, from the met requirements and the CV profile
Screening	Excluded for a missed must-have (`excluded`)	Rejected automatically	`screening_reject`	AI, from the missing must-haves, the CV profile and the CV skills
Screening	CV could not be read (`screening_failed`)	Rejected automatically	`cv_unreadable`	Fixed message
Interview	Report approved, decision **Reject**	Interviewer	`interview_reject`	AI, from the approved report
Interview	Report approved, decision **Hire**	Interviewer	`hire`	Fixed message

- Everyone not shortlisted counts as rejected. They can still be shortlisted until their email is sent; then shortlisting is locked for them. Shortlisting someone removes an email not sent yet.
- After the interview the decision is Hire or Reject (no Hold), only once the report is approved. It can change until the email is sent; a change removes the email written for the old decision.
- A screening rejection gets the `reject` decision when its email is sent.

2. Data flow



What the AI never receives: the candidate's name (also taken out of the CV summary) or email address, scores or ranks, other candidates, transcript quotes, the identity check, the interviewer's notes.

3. Workflow: LangGraph 5 (`backend/app/graphs/feedback/`)

One run per candidate, thread `feedback-{candidate_id}`, checkpointed in PostgreSQL so a waiting draft survives a restart.

Node	Type	What it does
`load_evidence`	State	Works out the email kind and loads the job-related points (`store.py`)
`use_template`	Rule	Fixed message for `hire` and `cv_unreadable`
`draft_feedback`	LLM	Writes the email in its own words, following the agreed order as a guide, and lists the ids of the points it draws on (`writing.py`)
`check_fairness`	Rule	Safety only: scores, other candidates, personal topics, personal data (emails, phones, links), promises, offer details, quotes, HTML and placeholders, and named skills or tools found in neither the candidate's points nor the job's requirements (`fairness.py`)
`review_email`	LLM	Respectful and kind? Anything the candidate's points do not support? Anything unfair by meaning? If it cannot run, the draft is marked *not checked*
`save_draft`	Output	Saves the draft, its warnings and its evidence; status `waiting_review`
`human_review`	HITL	`interrupt()`: approve, send anyway, edit, request changes, discard
`send_email`	MCP	`send_email` on the Google MCP server, backend only
`record_sent`	Output	Sent time, approver, `sent_anyway`; locks the decision
`mark_not_sent`	Error	Keeps the draft as `failed` and waits for Try again
`handle_error`	Error	Nothing to write from, or the AI is down on a first draft

Retries: a draft that breaks a rule or fails the review is rewritten with the problems named, up to `MAX_ATTEMPTS` (3). After the last try the problems stay as warnings; each names the exact words, which the screen highlights. An edit by the interviewer gets the same checks but is never rewritten (warnings only). If the AI is down during a requested rewrite, the old draft stays and waits again.

There are no limits on how many points the AI uses and no exact-word matching: it may rephrase, sum up and add encouraging advice. The order it is given, as a guide only: thanks → real strengths → sincere apology and the reason (mandatory requirements not found in the CV / preferred experience not found / growth areas after the interview / many strong applications when there is nothing to list) → why it matters for this role → warm closing.

After an interview, growth areas are named as the broader skill area they belong to, never the specific question or concept that was asked, so the interview questions stay with the team. The sentence on why the areas matter is written in the AI's own words from the job's requirements (never invented duties), keeping the meaning of the role level; preferred requirements missed at

screening are described as strengthening an application, never as a need. The signature is added by code.

Strengths: screening emails lead with the candidate's main area as the CV shows it, even when it is not the role's focus. The profile point `CV` is the CV summary, or the job titles in the CV when there is no summary; the candidate's name is taken out of it first, also when the CV spells it differently from the form. Interview emails lead with what the interview showed. When there are no strengths, the AI is told to leave that part out. Every exclusion reason becomes a growth area, including wordings other than a missing must-have or too few years. The greeting skips titles and short forms (Md., Dr.) and fixes all-capital names.

Role level (`templates.role_level`): internships and trainee roles → *entry*; 3+ required years → *senior* (needed "from the very first day"); otherwise *junior*. The level sets the meaning of the "why it matters" sentence, not its exact words.

4. Data model (migration `0016`)

`candidate_feedback` (one row per candidate, deleted with the candidate or the job): `kind`, `status` (`waiting_review`, `sending`, `sent`, `failed`, `discarded`), `written_by` (`ai` / `template`), `subject`, `body`, `evidence` (JSON, interviewer-only), `warnings` (JSON), `attempts`, `error_message`, `sent_anyway`, `approved_by`, `approved_at`, `sent_at`. The candidate's email address is not copied. Existing columns `candidates.decision`, `decided_by`, `decided_at` hold the decision.

5. API (interviewer only; candidates get 403)

Endpoint	Purpose
`PUT /candidates/{id}/decision`	Hire, reject, or take the decision back (`null`)
`POST /jobs/{id}/decisions`	Hire by rule (`top` or `min_score`), reject the other ranked candidates
`GET /jobs/{id}/feedback`	Writing or sending running, last run, emails to send per stage
`POST /jobs/{id}/feedback`	Write the emails of chosen candidates (background task)
`POST /jobs/{id}/feedback/send`	Send every passed draft of a stage (background task)
`GET /candidates/{id}/feedback`	One email with its evidence and warnings
`POST /candidates/{id}/feedback/review`	`approve`, `send_anyway`, `edit`, `request_changes`, `discard`

`CandidateSummary`, the ranked assessments and the assessment add `decision`, `feedback_status` (and `feedback_warnings`) for the screens.

6. Integration with the existing modules

- **Screening (F1/F2):** the final ranking shows each candidate's stage chip (*Not shortlisted* until the rejection email is sent, then *Rejected*; also the interview decision) and the email status; the

shortlist is locked once a rejection email is sent; the Feedback emails sub-tab lists everyone not shortlisted.

- **Assessment (F5/F6):** decisions need an approved report; the Assessments tab gets Decide by rule, Hire | Reject per row, the email column and a short summary in the opened row; the assessment page gets a Decision tab.
- **Google MCP server:** its `send_email` tool, kept until now for this feature, sends the approved email.
- **Background tasks:** writing and sending run like evaluations, one at a time, without blocking Fetch CVs or Screen CVs.
- **Deletion:** the email and the graph's saved state go with the candidate or the job.

7. Future work

Starting the emails from the AI Agent chat: a `ComposeFeedback` tool with an approval card that resumes this graph's `human_review`, and a read-only feedback status tool for the agent.